

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

CO1-AT2-Constraint-Based Problem Solving

Register Number	192311097
Name	DILLILOKESH D

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 20%

QUESTIONS:3

Total Marks:30

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- D1 cannot work Night shift
- D2 must work before D3 (Morning < Afternoon < Night)
- Only one doctor per shift
- D3 cannot work Morning
- All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a **grid environment (5×5)**. It must move from **Start (S)** to **Goal (G)**. Obstacles block certain paths.

Constraints:

- Movement allowed: Up, Down, Left, Right
- Cost of each move = 1
- Cannot pass through obstacles
- Use **Manhattan Distance heuristic**

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2

- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- Analyze all the given constraints and explain how they affect the robot's decision-making process.
- Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5
Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6

Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5
---------------	--	--	-------------------------------	---------------------------	-----

CO1-AT2: Constraint Based Problem Solving - Solutions

Question 1: Hospital Doctor Shift Assignment

Problem: Assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) with constraints:

- D1 cannot work Night shift
- D2 must work before D3 (Morning < Afternoon < Night)
- Only one doctor per shift
- D3 cannot work Morning
- All doctors must be assigned exactly one shift

Solution using Backtracking Search:

We'll assign shifts in order: Morning (M), Afternoon (A), Night (N)

Step 1: Assign Morning shift

- Try D1: Valid (D1 can do Morning)
□ State: [D1, _, _]
- Try D2: Valid (no constraints violated yet)
□ State: [D2, _, _]
- Try D3: INVALID (D3 cannot do Morning - constraint violation)

Step 2: Assign Afternoon shift (for each valid Morning assignment) Case 1:

Morning = D1

- Try D2: Check constraints:
 - D2 before D3? Not yet assigned D3, so OK for now
 - D1 not Night? D1 is Morning, so OK□ State: [D1, D2, _]
- Try D3: Check constraints:
 - D2 before D3? D2 is Morning, D3 would be Afternoon □ Morning < Afternoon, so D2 before D3 holds
 - D1 not Night? D1 is Morning, so OK□ State: [D1, D3, _]

Case 2: Morning = D2

- Try D1: Check constraints:
 - D2 before D3? D2 is Morning, D1 would be Afternoon □ We need to check if this violates D2 before D3

Actually, constraint is D2 must work BEFORE D3, meaning D2's shift < D3's shift. If D2 is Morning and D1 is Afternoon, we don't know where D3 will go yet.

So this is OK for now

- D1 not Night? D1 is Afternoon, so OK

□ State: [D2, D1, _]

- Try D3: Check constraints:
 - D2 before D3? D2 is Morning, D3 would be Afternoon □ Morning < Afternoon, so OK
 - D1 not Night? Not relevant here□ State: [D2, D3, _]

Step 3: Assign Night shift (for each valid partial assignment) Case 1.1:

[D1, D2, _]

- Only D3 left: Try D3
- Check constraints:
- D1 not Night? D1 is Morning ☐ OK
- D2 before D3? D2 is Afternoon, D3 would be Night ☐ Afternoon < Night ☐ OK
- D3 not Morning? D3 is Night ☐ OK
- ☐ VALID SOLUTION FOUND: [D1, D2, D3] = [Morning, Afternoon, Night]

Case 1.2: [D1, D3, _]

- Only D2 left: Try D2
- Check constraints:
- D1 not Night? D1 is Morning ☐ OK
- D2 before D3? D2 would be Night, D3 is Afternoon ☐ Night > Afternoon ☐ D2 is NOT before D3
- ☐ VIOLATION
- ☐ Backtrack

Case 2.1: [D2, D1, _]

- Only D3 left: Try D3
- Check constraints:
- D1 not Night? D1 is Afternoon ☐ OK
- D2 before D3? D2 is Morning, D3 would be Night ☐ Morning < Night ☐ OK
- D3 not Morning? D3 is Night ☐ OK
- ☐ VALID SOLUTION FOUND: [D2, D1, D3] = [Morning, Afternoon, Night]

Case 2.2: [D2, D3, _]

- Only D1 left: Try D1
- Check constraints:
- D1 not Night? D1 would be Night ☐ VIOLATION (D1 cannot work Night)
- ☐ Backtrack

Final Valid Schedules:

1. D1 ☐ Morning, D2 ☐ Afternoon, D3 ☐ Night
2. D2 ☐ Morning, D1 ☐ Afternoon, D3 ☐ Night

Both schedules satisfy all constraints.

Question 2: Robot Path Finding in Grid

Problem: Robot in 5x5 grid from Start (S) to Goal (G) with obstacles. Constraints:

- Movement: Up, Down, Left, Right
- Cost per move = 1
- Cannot pass through obstacles
- Use Manhattan Distance heuristic

Note: The problem mentions both BFS and Manhattan Distance heuristic, which is inconsistent (BFS is uninformed and doesn't use heuristics). Assuming the intent is to use A* search with Manhattan distance heuristic (which combines uniform cost with heuristic).

Solution using A* Search:

Since the specific obstacle layout wasn't provided in the problem statement, I'll demonstrate the methodology using a sample 5x5 grid. Let's assume:

- Grid size: 5x5 (coordinates 0-4 for rows and columns)
- Start (S): (0, 0) - top-left corner

- Goal (G): (4, 4) - bottom-right corner
- Obstacles: (1,1), (2,2), (3,1) - sample obstacles

A* Algorithm Components:

- $g(n)$ = cost from start to current node
- $h(n)$ = Manhattan distance to goal = $|x_goal - x_current| + |y_goal - y_current|$
- $f(n) = g(n) + h(n)$ = estimated total cost
- Priority queue ordered by $f(n)$ (lowest first)

Step-by-step Execution:

Initialization:

- Start node S(0,0): $g=0$, $h=|4-0|+|4-0|=8$, $f=0+8=8$
- Priority Queue: [(S, f=8)]
- Visited: {}

Step 1: Expand S(0,0)

- Remove S from queue (lowest f=8)
- Check if S is goal? No
- Generate neighbors:
 - Right: (0,1): Valid (not obstacle)
 $g=1$, $h=|4-0|+|4-1|=7$, $f=1+7=8$
 - Down: (1,0): Valid (not obstacle)
 $g=1$, $h=|4-1|+|4-0|=7$, $f=1+7=8$
- Add to queue: [(0,1,f=8), (1,0,f=8)] (order may vary)
- Visited: {S}

Step 2: Expand (0,1)

- Remove (0,1) from queue (f=8)
- Check if goal? No
- Generate neighbors:
 - Left: (0,0): Already visited
 - Right: (0,2): Valid
 $g=2$, $h=|4-0|+|4-2|=6$, $f=2+6=8$
 - Down: (1,1): OBSTACLE (skip)
 - Up: Out of bounds (skip)
- Add to queue: [(1,0,f=8), (0,2,f=8)]
- Visited: {S, (0,1)}

Step 3: Expand (1,0)

- Remove (1,0) from queue (f=8)
- Check if goal? No
- Generate neighbors:
 - Up: (0,0): Already visited
 - Down: (2,0): Valid
 $g=2$, $h=|4-2|+|4-0|=6$, $f=2+6=8$
 - Left: Out of bounds (skip)
 - Right: (1,1): OBSTACLE (skip)
- Add to queue: [(0,2,f=8), (2,0,f=8)]
- Visited: {S, (0,1), (1,0)}

Continue this process...

Note: The actual path will depend on the obstacle placement. The algorithm continues expanding nodes with lowest f-value until the goal is reached.

Key Points:

- Priority queue ensures we explore most promising paths first
- Manhattan heuristic is admissible (never overestimates) for grid with 4-direction movement
- When goal is reached, the path is optimal due to admissibility of heuristic
- Obstacles are skipped during neighbor generation

Question 3: Autonomous Rescue Robot in Disaster Environment

Problem: Rescue robot in partially observable, disaster-affected building grid.

Constraints:

- Movement: Up, Down, Left, Right
- Some cells blocked (obstacles)
- Limited sensing: can only observe adjacent cells
- Movement cost = 1, risky zones cost additional 2 (total 3)
- Must minimize total cost while ensuring safe navigation
- Must update knowledge dynamically (online search scenario)

a) Constraint Analysis and Impact on Decision-Making:**Partial Observability:**

- Robot cannot see entire map, only adjacent cells
- Impact: Cannot plan globally; must make decisions based on local information
- Requires belief state maintenance or online replanning as new information is sensed

Dynamic Obstacles/Unknown Terrain:

- Some paths blocked, but robot doesn't know which until adjacent
- Impact: Pre-computed paths may become invalid; need for replanning
- Requires robust replanning mechanisms when new obstacles are discovered

Variable Movement Costs:

- Normal movement = 1, risky zones = 3 (1 base + 2 risk penalty)
- Impact: Simple BFS/DFS insufficient; need cost-aware algorithms
- Risk-aware path planning: may detour around risky areas even if longer

Safety Requirement:

- Must ensure safe navigation (avoid known dangers)
- Impact: Cannot take paths through known hazardous areas
- May need to define safety thresholds or risk bounds

Online Nature:

- Must act while planning; cannot wait for complete knowledge
- Impact: Deliberation time affects performance; need anytime algorithms

b) Solution Approach: Online Search Agent with Real-Time A* (RTA*)**Why RTA*?**

- Combines benefits of heuristic search with online operation
- Handles partial observability by updating heuristic as knowledge grows
- Provides bounded suboptimality guarantees
- Computationally efficient for real-time operation

Steps:**1. Initialization:**

- Start with known map (start position and observable adjacent cells)
- Initialize heuristic values (e.g., Manhattan distance to goal)
- Set current state = start position

2. Local Real-Time Search (LRTA*)-style update:

- From current state, generate all possible actions (neighbors)
- For each action, simulate outcome and calculate cost:
 - Movement cost (1 or 3 for risky)
 - Heuristic estimate of resulting state to goal
- Select action with minimum total cost (movement + heuristic)
- Execute the selected action (move to chosen neighbor)
- Update heuristic for previous state based on experience:
 - $H(s) = \min_{\text{over_actions}} [\text{cost}(s,a,s') + H(s')]$

3. Perception and Model Update:

- After moving, sense adjacent cells (update map with new obstacle/free space info)
- If new obstacles detected in planned path, mark them and continue
- Heuristics may need adjustment if new information changes distance estimates

4. Termination:

- Stop when current state = goal state (survivor found)
- Optionally continue to refine path or search for additional survivors

Alternative: Online A* with D*-Lite principles

- More complex but handles changing costs better
- Maintains consistent heuristics and repairs paths efficiently

c) AI Concepts Demonstrated:

Intelligent Agent:

- Perceives environment through limited sensors (adjacent cells)
- Acts upon environment through movement actions
- Has goal: reach survivor at G
- Improves performance through learning (updating heuristics based on experience)

Rationality:

- At each step, chooses action that minimizes estimated total cost (path cost + heuristic)
- With perfect heuristic, this would be optimal; with learned heuristic, approaches optimality
- Bounded rationality due to computational constraints and partial observability

Environment Type:

- Partially Observable: Agent cannot see full state, only local observations
- Sequential: Current action affects future states and options
- Dynamic: Environment may change (new obstacles discovered/suspected)
- Stochastic: Sensor readings may have uncertainty (though not explicitly stated)
- Episodic: Not applicable - subsequent episodes depend on previous actions
- Continuous: State space is continuous in theory, but we discretize to grid cells

d) Justification for RTA* Approach:

Appropriateness for Partial Observability:

- Unlike offline algorithms (A*, UCS) that require complete map, RTA* works with current

knowledge

- Updates knowledge incrementally as sensors provide new information
- Heuristic values improve with experience, leading to better decisions over time

Efficiency in Real-Time:

- Only expands nodes in local neighborhood each step (constant time per step)
- Suitable for embedded systems with limited processing power
- Provides timely responses crucial for search-and-rescue scenarios

Handling Dynamic Costs:

- Naturally incorporates variable movement costs (1 vs 3) in cost calculation
- Can learn to avoid high-cost risky areas through experience
- Risk aversion emerges naturally incorporated directly into transition costs

Comparison to Alternatives:

- Pure Reactive agents (e.g., potential fields): May get trapped in local minima, no guaranteed progress
- Online DFS/BFS: Poor performance in large spaces, no heuristic guidance
- Periodic replanning with A*: Computationally expensive, may freeze robot while planning
- RTA*: Constant-time decision making, learns from experience, proven convergence properties

Theoretical Guarantees:

- RTA* with admissible heuristic converges to optimal path with sufficient trials
- In stationary environments, eventually behaves like A*
- In changing environments, continuously adapts to new information